INSTITUTO FEDERAL CATARINENSE – CAMPUS RIO DO SUL

CIÊNCIA DA COMPUTAÇÃO - PROGRAMAÇÃO DE ALTO DESEMPENHO

Professor: Rodrigo Curvêllo

Aluno: Rafael Colaço

Atividade: Aula 05 - #pragma omp critical, atomic, barrier e flush

1. Explique o que é race conditions.

A condição de corrida (race condition) acontece quando duas ou mais threads tentam acessar e modificar a mesma variável compartilhada na memória ao mesmo tempo sem nenhuma sincronização entre elas. Como não há controle sobre qual thread lê ou grava primeiro, uma thread pode acabar sobrescrevendo o valor que outra acabou de calcular, gerando resultados errados e diferentes a cada execução do programa.

2. Explique como funciona o #pragma omp critical.

A diretiva #pragma omp critical serve para criar uma região crítica no código onde apenas uma thread por vez pode entrar. Quando várias threads chegam nesse trecho durante a execução paralela, uma delas entra e executa o bloco enquanto as outras ficam esperando na fila até que a thread atual saia, evitando que duas threads alterem dados compartilhados simultaneamente.

3. Qual a diferença do #pragma omp critical para o #pragma omp single.

A diferença principal está em quantas threads vão executar aquele trecho de código. No #pragma omp critical, todas as threads do time vão passar pelo bloco e executá-lo, mas fazendo isso uma de cada vez em sequência. Já no #pragma omp single, apenas uma única thread do time executa o bloco (normalmente a primeira que chegar nele), enquanto todas as outras threads simplesmente pulam esse trecho.

4. Explique como funciona o #pragma omp atomic.

O #pragma omp atomic garante que uma operação simples em uma variável compartilhada, como um incremento ou uma soma, seja feita de forma inteira e indivisível na memória. Ele utiliza instruções atômicas do próprio processador para

travar apenas aquele endereço de memória específico durante a atualização, garantindo que nenhuma outra thread interfira no meio da operação.

5. Quais as diferenças entre #pragma omp critical e #pragma omp atomic.

O #pragma omp critical pode proteger um bloco inteiro de código com várias linhas, chamadas de funções e estruturas mais complexas, mas tem um custo maior de desempenho por usar travas de software que bloqueiam as outras threads. Por outro lado, o #pragma omp atomic funciona apenas para uma única linha com uma operação matemática ou de atribuição simples na memória, sendo bem mais rápido e leve porque usa suporte direto do hardware e bloqueia apenas o endereço que está sendo alterado.

6. Explique como funciona o #pragma omp barrier.

O #pragma omp barrier cria um ponto de parada obrigatória onde todas as threads precisam se encontrar antes do programa continuar. As threads que terminam sua parte do trabalho mais cedo ficam paradas na barreira esperando até que a última thread do time também chegue naquele ponto, e só então todas são liberadas juntas para seguir para o restante do código.

7. Explique como funciona o #pragma omp flush e cite os pontos implícitos que ele é chamado.

O #pragma omp flush serve para atualizar e sincronizar a memória entre as threads, garantindo que um valor modificado por uma thread em cache ou registrador seja realmente gravado na memória principal e que as outras threads leiam esse valor atualizado em vez de usarem uma cópia antiga. No OpenMP, o flush já acontece de forma implícita nas barreiras explícitas (#pragma omp barrier), na entrada e saída de regiões paralelas (#pragma omp parallel), na entrada e saída de regiões críticas (#pragma omp critical) e ordenadas (#pragma omp ordered), além do final das diretivas de divisão de trabalho como for, sections e single quando elas não usam a cláusula nowait.

8. Atualização Segura de Variáveis com #pragma omp critical

No programa abaixo (src/ex08.cpp), o vetor de inteiros é lido em paralelo com #pragma omp parallel for e o incremento da posição correspondente no histograma é protegido com #pragma omp critical para evitar condições de corrida.

```
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <omp.h>

int main() {
    int n = 10000;
    int vet[10000];
    int hist[10] = {0};

    srand(time(NULL));
    for (int i = 0; i < n; i++) {
        vet[i] = rand() % 10;
    }

    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        int val = vet[i];
        #pragma omp critical
        {
            hist[val]++;
        }
    }

    for (int i = 0; i < 10; i++) {
        printf("Valor %d: %d\n", i, hist[i]);
    }

    return 0;
}
```

9. Contagem Paralela com Atualização Atômica

No programa abaixo (src/ex09.cpp), o laço percorre o vetor em paralelo com #pragma omp parallel for e os contadores de números pares e ímpares são atualizados com #pragma omp atomic.

```
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <omp.h>

int main() {
    int n = 10000;
    int vet[10000];
    int pares = 0;
    int impares = 0;

    srand(time(NULL));
    for (int i = 0; i < n; i++) {
        vet[i] = rand() % 1000;
    }

    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        if (vet[i] % 2 == 0) {
            #pragma omp atomic
            pares++;
        } else {
            #pragma omp atomic
            impares++;
        }
```

```
    }

    printf("Pares: %d\n", pares);
    printf("Impares: %d\n", impares);

    return 0;
}
```

10. Sincronização de Threads com #pragma omp barrier

No programa abaixo (src/ex10.cpp), cada thread grava seu valor inicial na primeira etapa, aguarda todas as demais na barreira com #pragma omp barrier e depois realiza a segunda etapa somando seu elemento com o da thread vizinha.

```
#include <cstdio>
#include <omp.h>

int main() {
    int vet[4];
    int res[4];

    omp_set_num_threads(4);

    #pragma omp parallel
    {
        int id = omp_get_thread_num();

        vet[id] = (id + 1) * 10;
        printf("Etapa 1 - Thread %d gravou %d\n", id, vet[id]);

        #pragma omp barrier

        int prox = (id + 1) % 4;
        res[id] = vet[id] + vet[prox];
        printf("Etapa 2 - Thread %d somou %d + %d = %d\n", id, vet[id],
vet[prox], res[id]);
    }

    return 0;
}
```

11. Comunicação de Dados entre Threads com #pragma omp flush

No programa abaixo (src/ex11.cpp), a thread produtora gera o valor na variável compartilhada e usa #pragma omp flush para sincronizar a memória com as threads consumidoras, que aguardam a sinalização antes de lerem o valor atualizado.

```
#include <cstdio>
#include <omp.h>

int main() {
    int dado = 0;
    int pronto = 0;

    omp_set_num_threads(4);

    #pragma omp parallel shared(dado, pronto)
```

```
    {
        int id = omp_get_thread_num();

        if (id == 0) {
            dado = 50;
            #pragma omp flush(dado)
            pronto = 1;
            #pragma omp flush(pronto)
            printf("Produtora (thread %d) enviou: %d\n", id, dado);
        } else {
            while (!pronto) {
                #pragma omp flush(pronto)
            }
            #pragma omp flush(dado)
            printf("Consumidora (thread %d) leu: %d\n", id, dado);
        }
    }

    return 0;
}
```

12. Cálculo Paralelo de Pi com Redução e Sincronização

No programa abaixo (src/ex12.cpp), as threads geram pontos aleatórios no intervalo entre 0 e 1 pelo método de Monte Carlo, atualizam o total de pontos dentro do círculo com #pragma omp atomic e sincronizam com #pragma omp barrier antes do cálculo final de Pi.

```
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <omp.h>

int main() {
    int total = 10000000;
    int dentro = 0;
    double pi = 0.0;

    omp_set_num_threads(4);

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int nt = omp_get_num_threads();
        unsigned int seed = time(NULL) + id * 100;
        int qtd = total / nt;
        int cont_local = 0;

        for (int i = 0; i < qtd; i++) {
            double x = (double)rand_r(&seed) / RAND_MAX;
            double y = (double)rand_r(&seed) / RAND_MAX;

            if (x * x + y * y <= 1.0) {
                cont_local++;
            }
        }

        #pragma omp atomic
        dentro += cont_local;

        #pragma omp barrier
```

```
        #pragma omp single
        {
            pi = 4.0 * (double)dentro / (double)(qtd * nt);
            printf("Pi aproximado: %f\n", pi);
        }
    }

    return 0;
}
```